

Java 27, for Programmers New to Java

A self-paced curriculum for developers who already write code in another language and now need Java. It does not re-teach what a variable or a loop is. It teaches how Java does things, and where Java will surprise you.

Twenty-two modules. Each has a lesson, runnable examples whose comments explain the *why*, key takeaways, and homework with a reference solution.

Every example in this repository has been run on JDK 27. Every compiler error quoted in a lesson was produced by actually compiling the broken code, and every number was measured rather than recalled.

Türkçe: the whole curriculum is also available in Turkish under [tr/](#). Code, commands and compiler output stay in English there; [tr/CEVIRI-NOTLARI.md](#) explains why.

What "Java 27" means here

[JDK 27](#) reached General Availability on 15 September 2026. It is **not** an LTS release, and its finalised features are almost entirely runtime-level: G1 as the default collector, compact object headers, post-quantum TLS key exchange, flight recorder redaction. Nothing in that list changes how you write Java.

So this curriculum is honest about its own title:

JDK 27 is the runtime and toolchain. The language you learn is the stable Java accumulated through [JDK 25 LTS](#).

Module 22 covers what JDK 26 and 27 actually changed, why non-LTS releases matter less than the version number suggests, and how to read a JEP for yourself.

Getting started

The only requirement is JDK 27. No Maven, no Gradle, no third-party libraries anywhere in this repository.

```
java -version      # you want: java version "27"
```

Installing it

Platform	Command
macOS (Homebrew)	<code>brew install openjdk@27</code>
Linux (SDKMAN)	<code>sdk install java 27-open</code>
Windows (winget)	<code>winget install Microsoft.OpenJDK.27</code>
Any	download from Adoptium or Oracle

Pointing at it

```
# macOS
export JAVA_HOME=$(/usr/libexec/java_home -v 27)

# Linux
export JAVA_HOME=/usr/lib/jvm/jdk-27
export PATH="$JAVA_HOME/bin:$PATH"

# Windows PowerShell
$env:JAVA_HOME = "C:\Program Files\Microsoft\jdk-27"
$env:PATH = "$env:JAVA_HOME\bin;$env:PATH"
```

Your first run

```
java modules/01-getting-started/examples/HelloWorld.java
```

No compile step and no build file. Module 01 explains what is actually happening.

The curriculum

Work through them in order. Each module's prose assumes you have read the ones before it.

#	Module	What it covers
01	getting-started	JVM, JRE and JDK; running a file directly; reading a compiler error
02	variables-and-types	Primitives against references, autoboxing traps, <code>var</code> , text blocks
03	control-flow	No truthiness, switch expressions, exhaustiveness, labelled break
04	methods	Pass-by-value including for objects, overload resolution, varargs
05	classes-and-objects	The classic <code>main</code> , encapsulation, static factories, flexible constructors
06	inheritance-and-polymorphism	Dynamic dispatch, <code>abstract</code> against <code>final</code> , the <code>equals</code> / <code>hashCode</code> contract
07	interfaces	Capabilities, default methods, the diamond conflict, fragile base classes
08	records-and-enums	Records, compact constructors, shallow immutability, enums with behaviour
09	sealed-and-pattern-matching	Sealed types, record patterns, guards, why exhaustiveness is the payoff
10	exceptions	Checked against unchecked, try-with-resources, the three ways failures vanish
11	generics	Erasure and what it forbids, heap pollution, PECS, invariance
12	collections	Choosing an implementation, sequenced collections, why keys must not change
13	lambdas	Method reference forms, capture and effective finality, the standard interfaces
14	streams	Laziness, collectors, gatherers, and the parallel data race
15	optional	What it is for, <code>orElse</code> against <code>orElseGet</code> , five anti-patterns
16	files-and-io	<code>Path</code> against <code>Files</code> , path traversal, streaming, why not <code>java.io.File</code>
17	standard-library-tour	Strings, <code>java.time</code> , daylight saving, <code>Math</code> traps, reproducible randomness
18	concurrency-basics	Races and visibility as separate hazards, <code>synchronized</code> , atomics, executors
19	virtual-threads	Carriers and unmounting, the benchmark, pinning today, scoped values
20	packages-and-modules	The directory rule, the classpath, jars, and why <code>--add-opens</code> exists
21	testing-your-code	Why <code>assert</code> is not a testing tool; a test harness in 39 lines
22	whats-new	What JDK 26 and 27 really changed, LTS, and how to read a JEP

How to use this

Read the module README, run the examples, then do the homework **before** opening the solution.

```
modules/<nn>--<name>/
├─ README.md      the lesson, ending in Key Takeaways
├─ examples/      runnable files, heavily commented
├─ homework/      the assignment and its acceptance criteria

solutions/<nn>--<name>/
├─ reference solutions, kept out of the lesson tree on purpose
```

The solutions are for checking yourself against, not for reading first. Each one is commented to explain *why* it made the choices it did, so it is worth reading after you have your own answer, and close to worthless before.

Every homework has explicit acceptance criteria. If your version meets them, it is correct even where it differs from the reference.

Running examples

```
# an ordinary example
java modules/02-variables-and-types/examples/TheAutoboxingTrap.java

# one that needs assertions
java -ea modules/21-testing-your-code/examples/UsingAssertions.java

# one that needs a preview feature
java --enable-preview --source 27 modules/22-whats-
    new/examples/PreviewStructuredConcurrency.java
```

Some examples fail **on purpose**, because a compiler error you produced yourself teaches more than one quoted at you. Each declares what it expects in its first ten lines:

Marker	Meaning
(none)	Compiles and runs, exits 0
// EXPECT: compile-error	Must fail to compile; it is teaching a compile error
// EXPECT: runtime-error	Must compile, then crash at runtime
// EXPECT: compile-only	Must compile; not run on its own
// EXPECT: assertions	Run with <code>-ea</code> , so <code>assert</code> statements are live
// EXPECT: preview	Run with <code>--enable-preview --source 27</code>

Verifying everything

```
./scripts/verify-examples.sh # the whole repository
./scripts/verify-examples.sh modules/09-sealed-and-pattern-matching # one module
```

The script runs every example and every reference solution, checks each behaves as its marker declares, and refuses to run at all on a JDK other than 27, because an example failing for toolchain reasons teaches you nothing.

Current state of this repository:

Modules	22
Example files	105
Reference solutions	25
Files verified	130
Full verification time	about 70 seconds

PDFs

Every lesson and homework is also available as a PDF, plus the whole curriculum as a single 209-page book.

```
./scripts/build-pdfs.sh # both languages, 99 PDFs
./scripts/build-pdfs.sh --en # English only -> pdf/
./scripts/build-pdfs.sh --tr # Turkish only -> pdf-tr/
./scripts/build-pdfs.sh --book # just the combined books
```

Output	What
pdf/Java-27-Curriculum.pdf	the complete book, with a table of contents
pdf-tr/Java-27-Mufredat.pdf	the same book in Turkish
pdf/NN-<module>.pdf	one lesson
pdf/NN-<module>-homework.pdf	one homework
pdf/00-overview.pdf , 00-spec.pdf , 00-plan.pdf , 00-tasks.pdf	this file and the project documents

The script needs [pandoc](#) and a Chromium-based browser. Chrome is used as the PDF engine because pandoc's default requires a LaTeX installation, and Chrome renders these tables and code blocks correctly without one. Rebuild after editing any Markdown; the whole set takes about a minute.

Project documents

- [SPEC.md](#) — what this project set out to build, and its success criteria
 - [tasks/plan.md](#) — the implementation plan
 - [tasks/todo.md](#) — the task breakdown, linked to issues
-

A note on how this was built

Every factual claim in these lessons was produced rather than recalled. Compiler errors were generated by compiling the broken code. Benchmarks were measured. JEP numbers, release dates and LTS designations were read from openjdk.org.

That discipline caught real mistakes during writing, including a composition example that disproved its own claim, a bucket-index demonstration where the chosen numbers hid the bug entirely, and a "what's new" history that turned out to have two more rounds than assumed. Each correction is recorded in the git history.

If you find something here that is wrong, the JEP index outlives this document. Check the source.